

ASSIGNMENT-1

TITLE

Implementation of Different Types of Inheritance Using JavaScript

AIM

To implement and demonstrate different types of inheritance in JavaScript using classes, objects, prototypes, and mixins.

DESCRIPTION

Inheritance is an important concept in **Object-Oriented Programming (OOP)**. It allows a child class or object to acquire the properties and methods of a parent class or object. In JavaScript, inheritance can be implemented using the `extends`, `super`, `Object.create()`, and mixin techniques.

The major types of inheritance demonstrated using JavaScript are:

1. **Single Inheritance** – A child class inherits from one parent class.
2. **Multilevel Inheritance** – A class inherits from another derived class, forming a chain of inheritance.
3. **Hierarchical Inheritance** – Multiple child classes inherit from the same parent class.
4. **Multiple Inheritance** – JavaScript does not directly support multiple class inheritance, so mixins can be used to obtain features from multiple sources.
5. **Hybrid Inheritance** - It is a combination of two or more types of inheritance in a single program.
6. The `extends` keyword is used to create a relationship between classes, while the `super` keyword is used to access the parent class's

constructor or methods. JavaScript's prototype mechanism forms the foundation of its inheritance model.

Thus, inheritance helps in code reusability, reducing duplication, and organizing programs using object-oriented principles.

Single Inheritance:

Program:

```
// Parent Class
class Animal {
  constructor(name) {
    this.name = name;
  }

  eat() {
    console.log(this.name + " is eating.");
  }
}

// Child Class
class Dog extends Animal {
  bark() {
    console.log(this.name + " is barking.");
  }
}

// Creating object of Child Class
let dog = new Dog("Bruno");

// Calling inherited method
dog.eat();

// Calling child class method
dog.bark();
```

OUTPUT:

```
Command executed now and took 1.589s
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> node .\singl
Bruno is eating.
Bruno is barking.
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> █
```

Multilevel Inheritance:

Program:

```
// Grandparent Class
class Animal {
    eat() {
        console.log("Animal is eating.");
    }
}

// Parent Class
class Dog extends Animal {
    bark() {
        console.log("Dog is barking.");
    }
}

// Child Class
class Puppy extends Dog {
    play() {
        console.log("Puppy is playing.");
    }
}

// Creating object of Child Class
let puppy = new Puppy();

// Calling inherited methods
puppy.eat();
puppy.bark();

// Calling child class method
puppy.play();
```

Output:

```
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> node .\multilevel.js
Animal is eating.
Dog is barking.
Puppy is playing.
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> █
```

Hierarchical Inheritance:

Program:

```
// Parent Class
class Animal {
  eat() {
    console.log("Animal is eating.");
  }
}

// Child Class 1
class Dog extends Animal {
  bark() {
    console.log("Dog is barking.");
  }
}

// Child Class 2
class Cat extends Animal {
  meow() {
    console.log("Cat is meowing.");
  }
}

// Creating objects
let dog = new Dog();
let cat = new Cat();

// Dog methods
dog.eat();
```

```
dog.bark();

// Cat methods
cat.eat();
cat.meow();
```

Output:

```
● PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> node .\heir.js
Animal is eating.
Dog is barking.
Animal is eating.
Cat is meowing.
❖ PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> █
```

Multiple Inheritance:

Program:

```
// Parent 1
class Father {
  work() {
    console.log("Father is working.");
  }
}

// Parent 2
const Mother = {
  cook() {
    console.log("Mother is cooking.");
  }
};

// Child Class
class Child extends Father {
  study() {
    console.log("Child is studying.");
  }
}
```

```

    }
}

// Add Mother's methods
Object.assign(Child.prototype, Mother);

// Create object
let child = new Child();

// Calling methods
child.work();
child.cook();
child.study();

```

Output:

```

● PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> node .\multiple.js
  Father is working.
  Mother is cooking.
  Child is studying.
❖ PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> 

```

Hybrid Inheritance:

Program:

```

// Single Inheritance
class Animal {
  eat() {
    console.log("Animal is eating");
  }
}

class Dog extends Animal {
  bark() {
    console.log("Dog is barking");
  }
}

```

```
let dog = new Dog();
dog.eat();
dog.bark();

// Multilevel Inheritance
class Puppy extends Dog {
  play() {
    console.log("Puppy is playing");
  }
}

let puppy = new Puppy();
puppy.eat();
puppy.bark();
puppy.play();

// Hierarchical Inheritance
class Cat extends Animal {
  meow() {
    console.log("Cat is meowing");
  }
}

let cat = new Cat();
cat.eat();
cat.meow();

// Multiple Inheritance using Mixin
const Mother = {
  cook() {
    console.log("Mother is cooking");
  }
};

class Child extends Animal {
  study() {
    console.log("Child is studying");
  }
}
```

```

}

Object.assign(Child.prototype, Mother);

let child = new Child();
child.eat();
child.cook();
child.study();

// Hybrid Inheritance
class Student extends Animal {
  study() {
    console.log("Student is studying");
  }
}

class Boy extends Student {
  play() {
    console.log("Boy is playing");
  }
}

class Girl extends Student {
  dance() {
    console.log("Girl is dancing");
  }
}

let boy = new Boy();
boy.eat();
boy.study();
boy.play();

let girl = new Girl();
girl.eat();
girl.study();
girl.dance();

```

Output:


```
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> node .\hybrid.js
Animal is eating
Student is studying
Boy is playing
Animal is eating
Student is studying
Girl is dancing
PS C:\Users\reddy\OneDrive\Desktop\assignment1-fs> 
```